

HandsMenThreads

Salesforce CRM

Project Documentation

Project Title	HandsMenThreads – Salesforce CRM Implementation
Platform	Salesforce Developer Edition
Document Type	Project Documentation(Phase Wise)
Prepared By	Raam Prakash S
Submission Date	September 2026

Project Overview

This project delivers a purpose-built Salesforce CRM for HandsMenThreads, a men's fashion label that had outgrown manually tracking its customers, orders and stock across scattered spreadsheets. Customer, product and order data now sit inside a single Salesforce org, and much of the routine work — sending order confirmations, adjusting stock levels, moving a customer up a loyalty tier — runs on its own without manual intervention.

The build combines Salesforce's declarative tools (Custom Objects, Flows, Validation Rules) with targeted Apex code for the handful of tasks clicks alone couldn't handle, primarily calculating order totals and keeping stock and loyalty status in sync in real time. All of it is wired into one Lightning app, so the Customer, Order, Product, Inventory, Campaign and Reports tabs sit together rather than being scattered across the org.

Objectives

The underlying goal was straightforward: remove the manual, error-prone pieces of running HandsMenThreads and give the team a clearer, shared view of sales, stock and customer activity. That translated into a short list of concrete targets for the system:

- Automate repetitive tasks — order confirmation emails, loyalty tier updates, low-stock warnings — so nobody has to remember to trigger them manually.
- Prevent bad data from entering the system at the source, mainly through validation on emails and stock levels.
- Give sales, inventory and marketing visibility into stock, orders and customer activity without having to ask around.
- Keep every team working from the same underlying data instead of separate personal records.
- Make the customer experience feel more personal through the loyalty program and automated touchpoints.

Phase 1: Requirement Analysis & Planning

Understanding Business Requirements

Before opening Salesforce, time went into understanding what was actually breaking down in the manual process. As HandsMenThreads grew busier, tracking customers and orders in spreadsheets meant totals had to be calculated by hand, there was no reliable way to catch low stock before it became a problem, and customers received no confirmation once an order was placed. What the business needed was a single home for customer and product data, automatic order totals, a hard stop on stock going negative, and a way to keep customers informed without extra manual effort.

Defining Project Scope and Objectives

With that understanding in place, the project was scoped around a few core building blocks rather than attempting everything at once:

- Custom objects for Customers, Products, Orders, Inventory and Marketing Campaigns, forming the backbone of the data model.
- Automation for order confirmations, stock deduction and loyalty status — the three areas that were most costly to handle manually.
- Validation on customer emails and product stock so invalid data is rejected before it's saved.
- Separate profiles and roles for Sales and Inventory, reflecting that each team needs a different view of the data.
- A single Lightning app tying everything together, instead of a collection of disconnected tabs.

Data Model and Security Model

The data model is built on four custom objects, plus a dedicated Inventory object that tracks stock separately from the product record:

- **HandsMen Customer (Customer__c)** stores Name, Email, Phone, Address and Loyalty_Status__c.
- **HandsMen Product (Product__c)** stores Name, Description, Unit_Price__c and Stock__c.
- **HandsMen Order (Order__c)** captures the order date, Quantity__c and Total_Amount__c, and links to both a Customer and a Product.
- **Marketing Campaign (Marketing_Campaign__c)** tracks promotions and marketing activity.
- **Inventory** records stock quantity per product/warehouse and feeds the low-stock alerts.

Security is handled in two layers. Profiles determine what a user can access at all — which objects, fields and pages are visible to them — while roles control how much of the data within that access they can see. A Sales Manager role sits above Sales Representative in the hierarchy, so a manager can see the whole team's orders while a rep sees only their own.

Phase 2: Salesforce Development – Backend & Configurations

Setting Up the Environment

A free Salesforce Developer Edition org was set up at developer.salesforce.com/signup, and all building, testing and screenshots for this project were carried out inside that org.

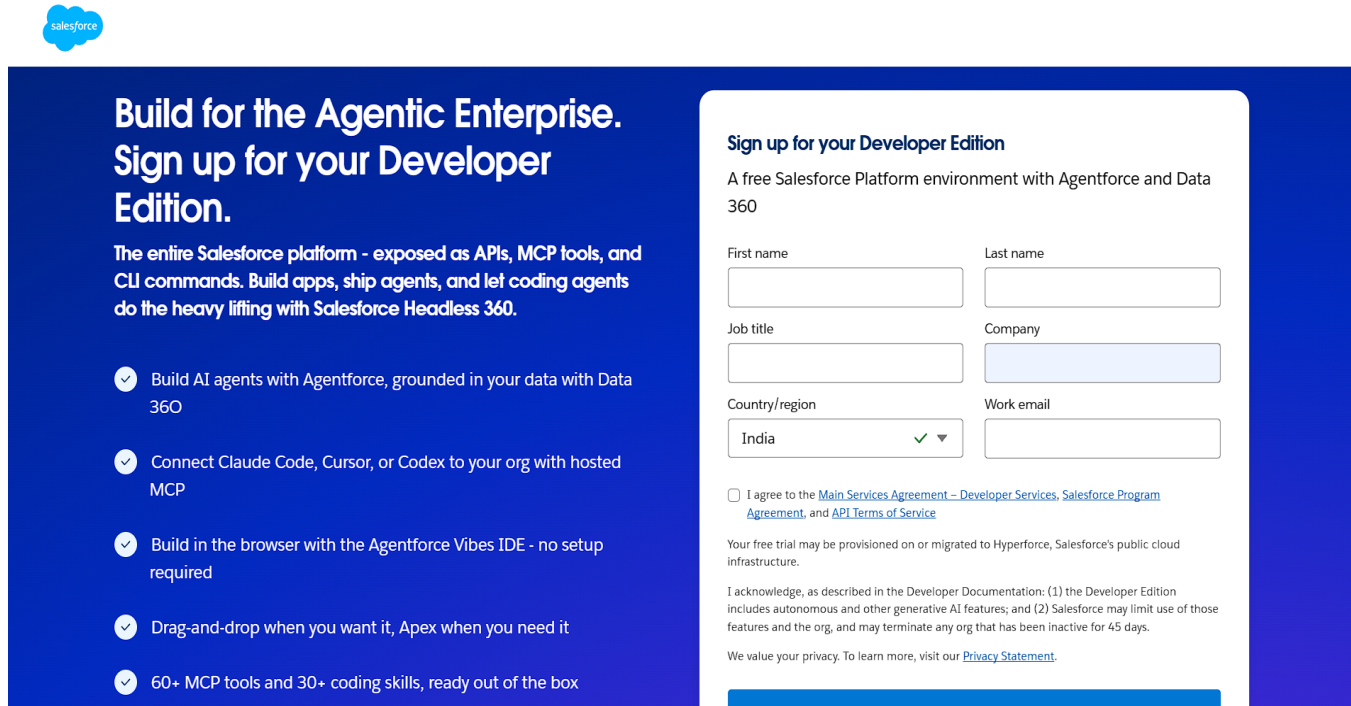
The image shows the Salesforce Developer Edition sign-up page. On the left, a blue banner with white text reads: "Build for the Agentic Enterprise. Sign up for your Developer Edition. The entire Salesforce platform - exposed as APIs, MCP tools, and CLI commands. Build apps, ship agents, and let coding agents do the heavy lifting with Salesforce Headless 360." Below this, five bullet points with checkmarks list benefits: "Build AI agents with Agentforce, grounded in your data with Data 360", "Connect Claude Code, Cursor, or Codex to your org with hosted MCP", "Build in the browser with the Agentforce Vibes IDE - no setup required", "Drag-and-drop when you want it, Apex when you need it", and "60+ MCP tools and 30+ coding skills, ready out of the box". On the right, a white sign-up form titled "Sign up for your Developer Edition" contains fields for "First name", "Last name", "Job title", "Company", "Country/region" (with a dropdown menu showing "India"), and "Work email". Below the form, there is a checkbox for "I agree to the Main Services Agreement – Developer Services, Salesforce Program Agreement, and API Terms of Service", followed by a disclaimer about the trial and a link to the "Privacy Statement". A blue "Sign Up" button is at the bottom right of the form.

Fig 2.1: Salesforce Developer Edition Sign-Up

Objects, Fields, Validation Rules and Automation

The four custom objects from Phase 1 (Customer__c, Product__c, Order__c, Marketing_Campaign__c) were configured first, each with its own custom tab so records can be created and browsed directly from the app rather than through Setup.

Two validation rules enforce data quality. One sits on the product record and prevents Stock__c from ever being saved below zero, which stops an order from going through for more stock than actually exists. The other sits on the customer record and requires the email field to contain "@gmail.com" before saving, keeping customer contact details in a consistent format.

Automation was built as three Flows in Flow Builder rather than as pure Apex, since Flows are easier to review and adjust later:

- An order confirmation flow, triggered when a HandsMen Order record is created, that emails the customer as soon as their order is placed.
- A low-stock alert flow, triggered on the Inventory object, that emails the inventory manager once a product's stock drops below the threshold.
- A scheduled flow that runs daily, loops through every customer, checks their cumulative purchases, and places them into the correct loyalty tier.

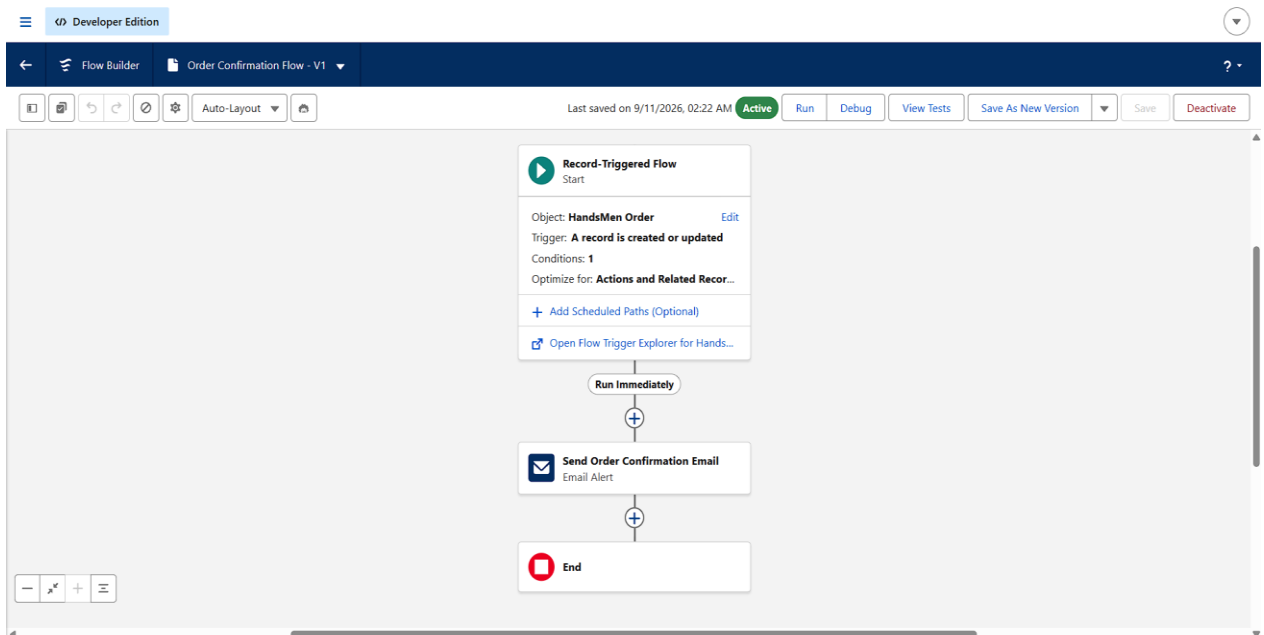


Fig 2.2: Order Confirmation Flow (Record-Triggered)

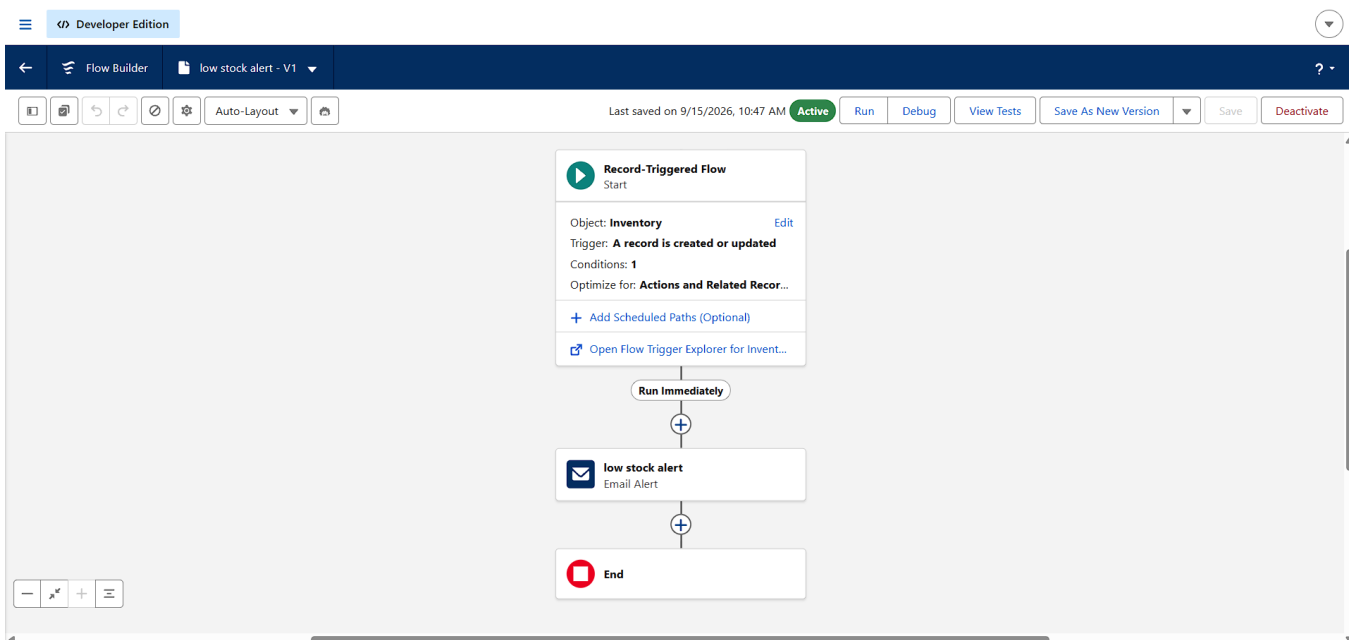


Fig 2.3: Low Stock Alert Flow (Record-Triggered on Inventory)

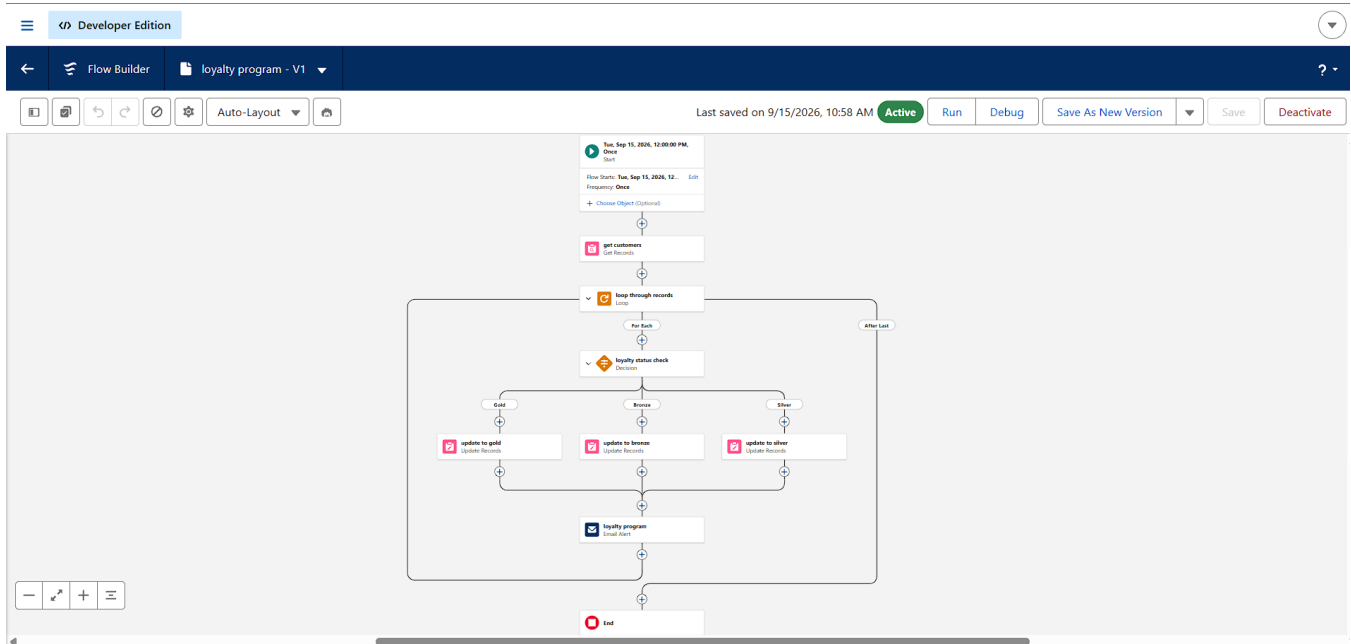


Fig 2.4: Scheduled Loyalty Status Update Flow (Daily, Schedule-Triggered)

Apex Classes & Triggers

A small number of requirements couldn't be met through clicks alone, so three Apex triggers were written to cover them. None of the logic needed to run asynchronously — no batch or queueable classes were required — since each trigger only performs a small amount of work at the moment a record is saved.

- **OrderTrigger** calculates `Total_Amount__c` on an order from the quantity and the product's unit price, removing the need for anyone to work it out manually.
- **ProductStockUpdateTrigger** deducts the ordered quantity from the product's stock once an order is confirmed, relying on the stock validation rule to stop stock from going negative.
- **CustomerLoyaltyUpdateTrigger** recalculates a customer's `Loyalty_Status__c` the moment their purchase total changes, so the update doesn't have to wait for the nightly scheduled flow.

Phase 3: UI/UX Development & Customization

Building the App in App Manager

A Lightning app named "HandsMen Threads" was assembled through App Manager, bringing the Customer, Orders, Products, Inventory, Reports and Dashboards tabs together in one place, so the team opens a single app instead of hunting for tabs individually.

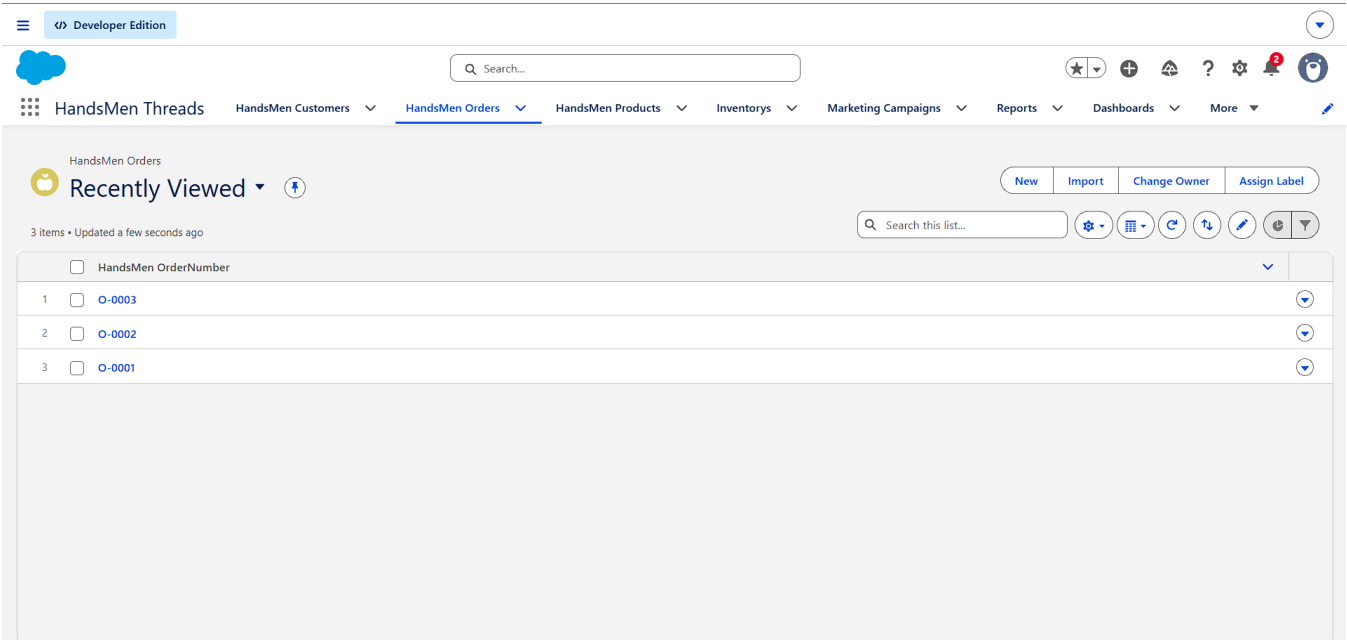


Fig 3.1: Custom "HandsMen Threads" Lightning App

Page Layouts & Record Creation

Page layouts were kept close to standard for each object, with fields arranged so the most important ones — name, contact details, stock, order total — are visible without scrolling. A few of the record pages currently in use are shown below.

Developer Edition

HandsMen Threads

HandsMen Customers

Recently Viewed

2 items • Updated a few seconds ago

	HandsMen Customer Name
1	Rahul
2	Raam

New HandsMen Customer

* = Required Information

Information

* HandsMen Customer Name [↻](#)
Raam Prakash

Email [↻](#)
raamprakash776@gmail.com

Phone

Loyalty Status [↻](#)
Gold

FirstName [↻](#)
Raam

LastName [↻](#)
Prakash

Total Purchases

Owner [↻](#) RAAM PRAKASH S

[Cancel](#) [Save & New](#) [Save](#)

Fig 3.2: Customer Record Creation (HandsMen Customer)

Developer Edition

HandsMen Threads

HandsMen Customers

HandsMen Orders

HandsMen Products

Inventories

Marketing Campaigns

Reports

Dashboards

More

HandsMen Products

Recently Viewed

2 items • Updated a few seconds ago

[New](#) [Import](#) [Change Owner](#) [Assign Label](#)

Search this list...

	HandsMen Product Name
1	Pant
2	Shirt

Fig 3.3: Product List View (HandsMen Products)

New HandsMen Order

* = Required Information

Information

HandsMen OrderNumber

Owner: RAAM PRAKASH S

Customer: Raam

Status: Confirmed

Quantity: 600

Total Amount:

* Customer Email: raamprakash776@gmail.com

Product: Search HandsMen Products...

Buttons: Cancel, Save & New, Save

Fig 3.4: New Order Creation Page (HandsMen Orders)

New Inventory

* = Required Information

Information

Inventory Number

* Product: Shirt

Stock Quantity: 10,000

Warehouse:

Buttons: Cancel, Save & New, Save

Fig 3.5: Inventory Record

User Management

To confirm the multi-user side worked as intended, a couple of test users were created and assigned to different profiles and roles — one for Sales, one for Inventory — so each user sees only what's relevant to their job.

Developer Edition

Search Setup

Setup

Home

Object Manager

user

Users

Analytics Groups

Permission Set Groups

Permission Sets

Profiles

Public Groups

Queues

Roles

User Management Settings

Users

Feature Settings

Data.com

Prospector Users

Service

Embedded Service

Enhanced Chat User Verification

SETUP

Users

All Users

On this page you can create, view, and manage users.

To get more licenses, use the Your Account app. Let's Go

View: All Users

Edit | Create New View

New User

Reset Password(s)

Add Multiple Users

Action	Full Name	Alias	Username	Role	Active	Profile
☐ Edit	Chatter Expert	Chatter	chatty.00df00000foeazea2.8ktnpexizql@chatter.salesforce.com		☒	Chatter Free User
☐ Edit	EPIC_OrgFarm	OEPIQ	epic.dacc4dc916a@orgfarm.salesforce.com		☒	System Administrator
☐ Edit	Mikaelson_Kol	knika	raampc7862@gmail.com	Inventory	☐	Platform 1
☐ Edit	Mikaelson_Niklaus	nniki	raampc7861@gmail.com	Sales	☒	Platform 1
☐ Edit	PRAKASH_S_RAAM	230	230791.cd.73cd39040f96@agentforce.com		☒	System Administrator
☐ Edit	Scott_Mark	mscot	raampc7863@gmail.com	Marketing	☒	Platform 1
☐ Edit	User_Integration	integ	integration@00df00000foeazea2.com		☒	Analytics Cloud Integration User
☐ Edit	User_Security	sec	insightssecurity@00df00000foeazea2.com		☒	Analytics Cloud Security User

New User

Reset Password(s)

Add Multiple Users

Fig 3.6: User Detail Page Showing Role and Profile Assignment

Reports and Dashboards

The app already includes a Reports tab and a Dashboards tab, giving a starting point for tracking sales and stock. Detailed dashboards haven't been built out yet — that's included under future enhancements at the end of this document.

Phase 4: Data Migration, Testing & Security

Data Loading

For this build, sample Customer, Product, Order and Inventory records were entered directly through the app UI, since the volume didn't justify anything more elaborate. If this were rolled out against the brand's real customer and product lists, the Data Import Wizard would handle a smaller, guided import, while Data Loader would be the better fit for a larger CSV-based migration.

Field History, Duplicate and Matching Rules

Loyalty_Status__c and Stock__c are the two fields most worth tracking, so enabling Field History Tracking on them would log who changed what and when — useful for tracing back what happened if a loyalty tier or stock figure ever looks wrong. Duplicate and matching rules on the Customer object, matched on email and phone, prevent the same person from being added twice by accident.

Profiles, Roles and Sharing

The Standard User profile was cloned to create a "HandsMen Sales Profile" tailored to what a sales rep actually needs, rather than granting everyone the same broad permissions. Roles — Sales Manager, Inventory Manager, and so on — sit in a hierarchy so a manager automatically sees the records of the people below them without anything needing to be shared explicitly. Where someone needs slightly more access than their profile provides, without changing that profile for everyone else, a permission set covers the gap instead. Sharing rules are kept to a minimum for now, since org-wide defaults cover most cases, but they would come into play if, for example, Inventory needed visibility into Order records they don't currently see.

Testing

Each of the three Apex triggers has a corresponding test class, covering both the positive path (an order saves cleanly and the total, stock and loyalty status all update correctly) and the negative path (attempting to oversell stock, or saving a customer with an invalid email), confirming that the validation rules genuinely block bad data rather than merely appearing to.

Beyond the automated tests, the scenarios below were run manually inside the org to confirm each piece of automation behaves as expected end-to-end.

Feature	Test Scenario	Expected Result
Customer Email Validation Rule	Enter an email without "@gmail.com"	Save is blocked with an error message
Order Total Trigger	Create an order for 1 unit at \$500	Total_Amount__c auto-calculates to \$500
Email Notification Flow	Save a new confirmed order	Order confirmation email is sent to the customer
Stock Deduction Trigger	Confirm an order	Product Stock__c reduces by the ordered quantity
Product Stock Validation Rule	Order more units than available stock	Save is blocked, preventing overselling

Feature	Test Scenario	Expected Result
Low Stock Alert Flow	Inventory quantity drops below threshold	Low-stock alert email sent to Inventory Manager
Loyalty Status Trigger / Scheduled Flow	Customer's cumulative purchases cross a threshold	Loyalty_Status__c updates (e.g., to "Bronze")

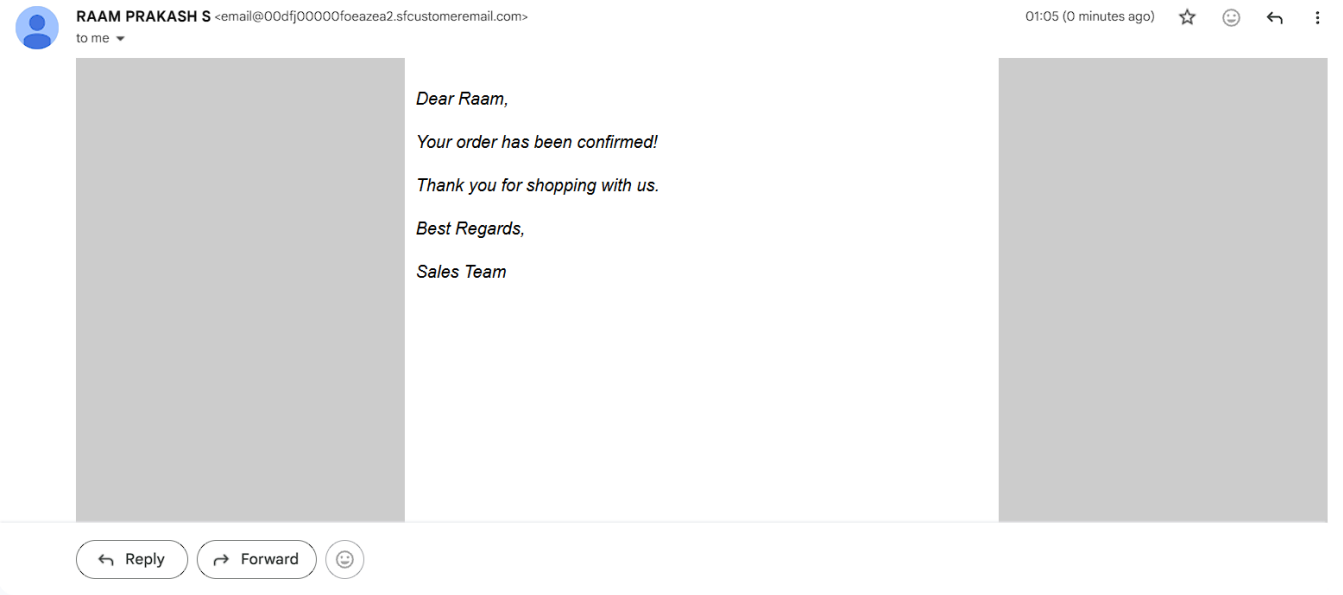


Fig 4.1: Order Confirmation Email — Output of Email Notification Flow

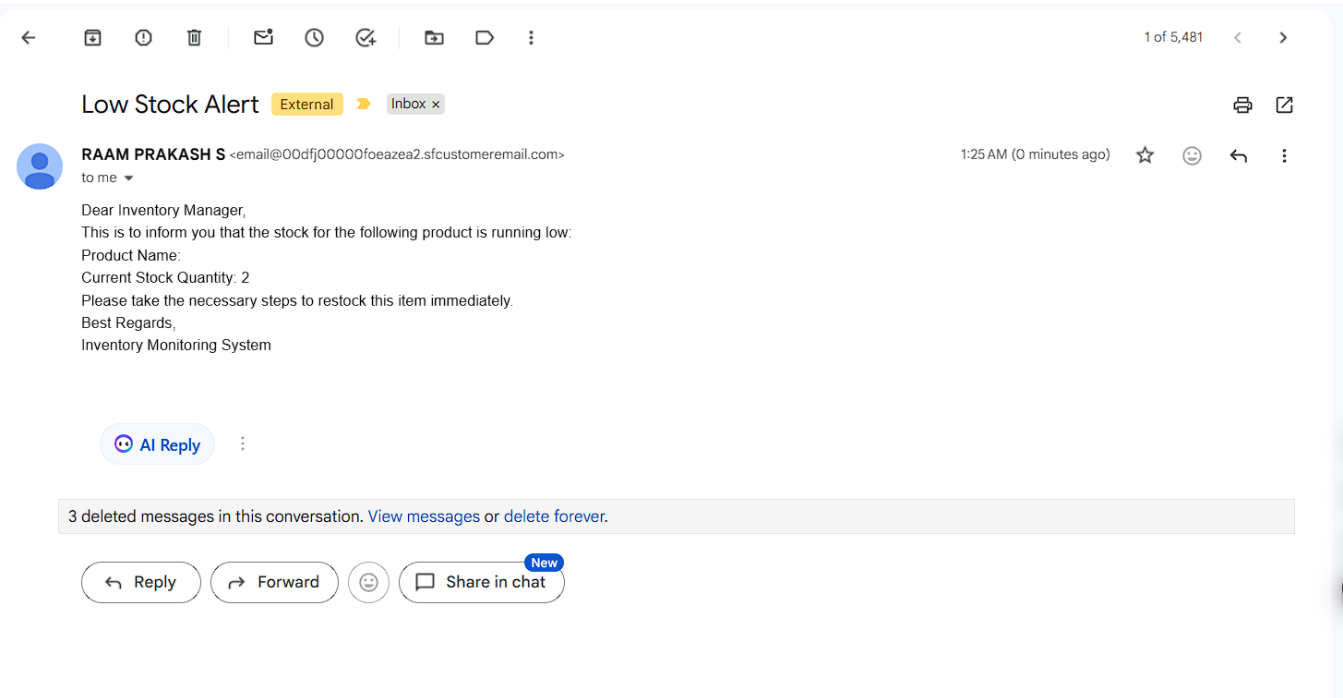


Fig 4.2: Low Stock Alert Email — Output of Stock Update Flow

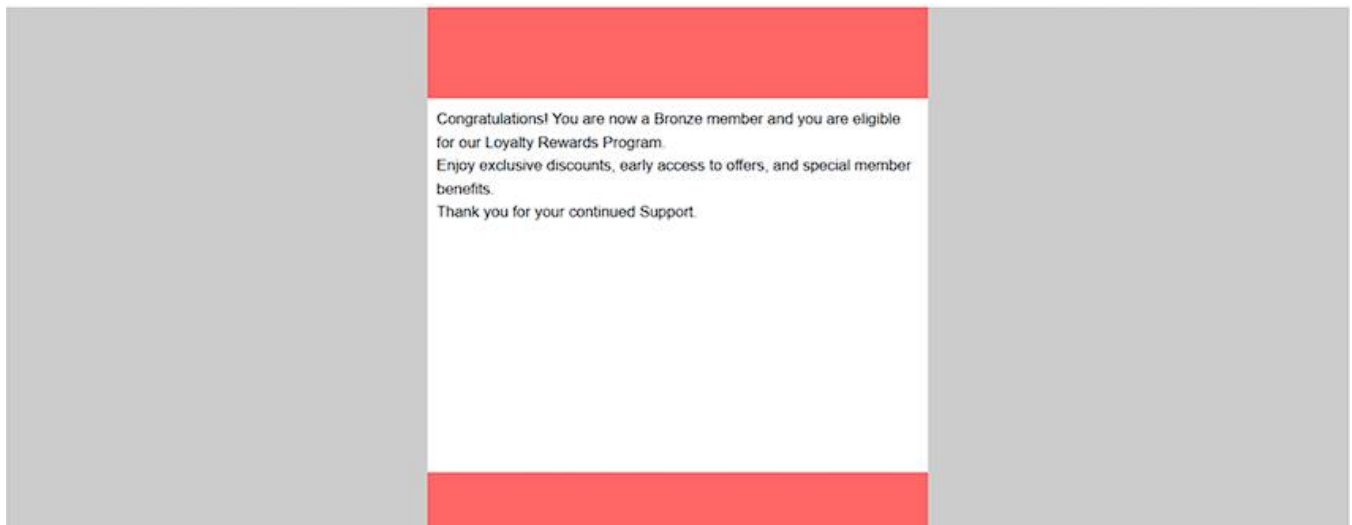


Fig 4.3: Loyalty Status Update Email — Output of Loyalty Status Trigger / Scheduled Flow

Phase 5: Deployment, Documentation & Maintenance

Deployment

Everything for this project was built and tested inside a single Developer Edition org, so no movement between environments was required. If this were promoted into a sandbox or production org, it would be deployed using change sets, moved roughly in dependency order — objects and fields first, then validation rules and flows, then the Apex classes and triggers along with their test coverage — validating each change set in the target org before deploying it for real.

Maintenance & Monitoring

Ongoing maintenance mainly involves periodically checking the flows' run history to confirm the order confirmation, low-stock alert and loyalty flows are still firing correctly, and revisiting the validation rules if business rules change — for instance, if customers should be allowed to sign up with non-Gmail addresses. Field history on the key fields provides a way to look back if something appears off.

Troubleshooting

When something doesn't fire as expected, the first place to check is the flow's run history, or the Apex debug logs for trigger-related issues, to see exactly where execution stopped. It's also worth checking whether a validation rule is quietly blocking the save before assuming the automation itself is broken. During testing, the low-stock alert email landed in spam on a couple of occasions, which turned out to be a sender/deliverability setting rather than an issue with the flow itself — worth keeping in mind if alerts seem to go missing.

Conclusion

Overall, this project moved HandsMenThreads' customer, order, inventory and campaign data off spreadsheets and into a single Salesforce org, with the previously tedious parts — totalling orders, deducting stock, sending confirmations, updating loyalty tiers — now happening automatically rather than manually. There's still considerable room to build on it, but even at this stage it removes a meaningful amount of manual work and gives a clearer view of sales and inventory activity than a set of spreadsheets ever could.

Future Enhancements

A few directions worth pursuing if this project continues beyond its current scope:

- A self-service customer portal, so customers can log in to check past orders and loyalty points without needing to ask.
- A mobile app built with the Salesforce Mobile SDK, aimed at in-store staff who need to check stock or place an order without a desktop.
- Proper sales and inventory dashboards, since the Reports/Dashboards tabs are set up but not yet built out.
- Einstein-based product recommendations, suggesting items based on a customer's purchase history.
- WhatsApp or SMS integration for order and shipping updates, since not everyone checks email regularly.
- A basic chatbot to handle common customer questions without needing a person to answer every time.